Proposal, version 3

A simpler way to score a run (`sec:pf:measures`)

For review before anything is applied

5 October 2026

Version 2 tried to repair the current scoring section one problem at a time. Version 3 replaces the scoring itself with a smaller scheme taken from the lifelong MAPD literature. Version 2 is kept as `scoring_proposal_v2.pdf` in the same folder. Nothing in the thesis has been changed, and the figures are drafts in `scoring_proposal/`.

Contents

1 Why change the scoring **1**

2 The principles behind the new scheme **2**

3 The four questions **2**
- 3.1 Question 1. Does the fleet keep up? . . . 3
- 3.2 Question 2. How fast is a task served? . . . 4
- 3.3 Question 3. Why did the result change? . . . 4
- 3.4 Question 4. What does deciding cost? . . . 7

4 The research question, formally **7**

5 What the literature measures **7**

6 What is removed and what it touches **8**

7 What happens to the version 2 decisions **9**

8 Your decisions **9**

1 Why change the scoring

The main test cannot be computed. The research question is formalised in `eq:rqformal` as a comparison of $\mathcal{J}$ under adaptive and fixed storage. $\mathcal{J}$ adds five quantities with weights q_{svc}, q_{mov}, q_{wait}, q_{conc} and q_{back}. No chapter gives these weights a value, and the Method's parameter tables do not list them even as TBD. The central result therefore rests on five numbers that do not exist.

Any choice of weights would be arbitrary. The section keeps runtime out of $\mathcal{J}$ because seconds cannot be traded against timesteps. The same holds inside $\mathcal{J}$. Service time is in timesteps, movement cost in cost units, backlog in tasks, and concentration is a fraction. Each weight sets an exchange rate between two units that the thesis would have to defend.

Lifelong MAPD scores with much less. The paper that defines lifelong MAPD, Ma et al. (2020), judges an algorithm by one number, the service time. It accepts a run only if the service time stays bounded, and reports runtime per timestep next to it. Section 5 lists what each paper in `thesis-progress/papers/` measures.

Throughput and backlog answer one question. Tasks arrive at a fixed rate λ_{task}. While the fleet keeps up, it finishes tasks as fast as they arrive, so throughput equals the arrival rate and the backlog stays small. Throughput and backlog only depart from that when the fleet falls behind. Together they answer whether the fleet keeps up. They are not two separate scores.

2 The principles behind the new scheme

1. **One primary result.** Mean service time decides whether adaptive storage beats fixed storage. No weighted sum is used.
2. **Every other measure answers a named question.** Each one either checks that a run is valid or explains why the primary result changed. None of them is traded against service time.
3. **Borrowed measures are cited and not claimed.** A measure taken from a paper is used as that paper defines it, with the citation, and is not presented as a contribution of the thesis. Where the thesis adapts a measure, the change is stated, and only the change is the thesis's own responsibility. A measure with no source in the literature is marked as the thesis's own.
4. **Each measure has the same layout.** The question it answers, one sentence saying what it counts, the formula, a worked number, its source, and its role.
5. **Every worked number comes from the same two examples.** These are the timeline of tasks τ_1 to τ_4 (Figure 1) and the warehouse with agents a_1 to a_4 (Figure 2), both drawn on bases the thesis already uses.

3 The four questions

A run is scored by four questions, asked in this order.

Question	Measure	Role
1. Does the fleet keep up?	throughput against arrival rate, backlog	check
2. How fast is a task served?	mean service time $\bar{\zeta}_T$	**primary result**
3. Why did it change?	blocked time, crowding, traffic share, movement cost	explanation
4. What does deciding cost?	runtime per timestep κ_T	reported

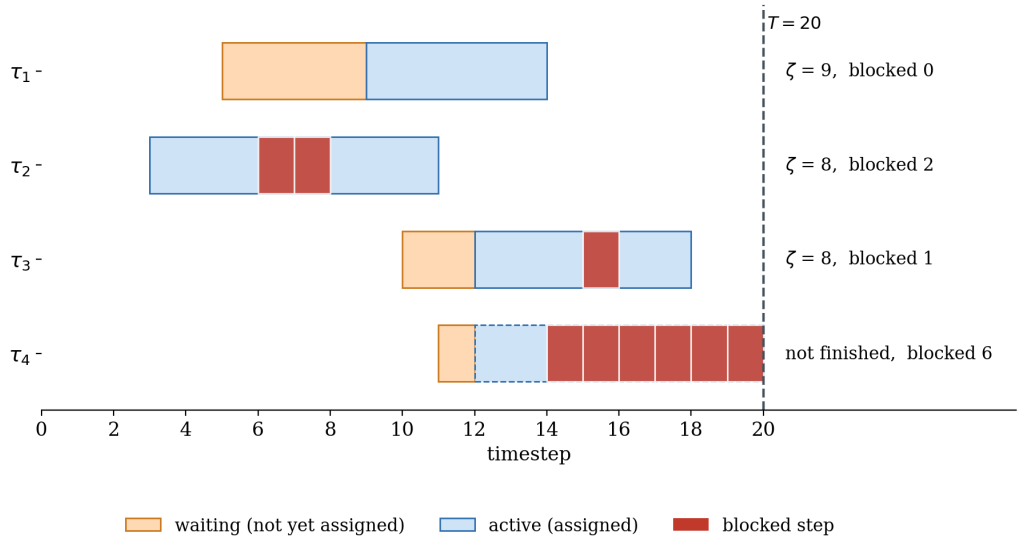


Figure 1: Four tasks over a horizon $T = 20$. τ_4 is stuck from $t = 14$ and never finishes. This is the worked example for questions 1 to 3.

3.1 Question 1. Does the fleet keep up?

Throughput against arrival rate, and backlog

Question. Is the fleet finishing tasks as fast as they arrive?

What it counts. Throughput is the number of tasks finished per timestep. The fleet keeps up when throughput stays close to the rate at which tasks arrive. The backlog, the number of tasks released but not yet finished, is logged at every timestep and plotted, so a run that falls behind can be seen.

Formula.

$$\Lambda_T = \frac{|\mathcal{C}_T|}{T}, \qquad \text{the run keeps up if } \Lambda_T \geq f_{\text{up}}\,\lambda_{\text{task}}, \qquad B_t = |\mathcal{Q}_t| + |\mathcal{B}_t|.$$

Worked number. In Figure 1, four tasks are released and three finish within $T = 20$. Throughput is $3/20 = 0.15$ per timestep against $4/20 = 0.20$ released, a ratio of 0.75. The backlog ends at $B_{20} = 1$, the stuck task τ_4. Without τ_4 the ratio would be 1 and the backlog 0.

Source. Ma et al. (2020) count a lifelong MAPD instance as solved only if the service time of all tasks stays bounded, which is the same as keeping up. Throughput as completed tasks per unit time is the pick rate that Krnjaic et al. (2022) optimise. *Own part:* the ratio test and its threshold f_{up}.

Role. A check, applied before anything else. A run that does not keep up is reported as such, and its service time is not compared, because service time only counts finished tasks.

3.2 Question 2. How fast is a task served?

Mean service time $\bar{\zeta}_T$, the primary result

Question. On average, how long does a task take from its release to its delivery?

What it counts. For each finished task, the number of timesteps between its release and its delivery, averaged over the finished tasks. The wait before assignment counts too, because the order is waiting during that time as well (`fig:lifecycle`).

Formula.

$$\bar{\zeta}_T = \frac{1}{|\mathcal{C}_T|} \sum_{\tau_j \in \mathcal{C}_T} \zeta_j, \qquad \zeta_j = t_j^{\text{finish}} - t_j^{\text{released}} \quad (\texttt{eq:servicetime}).$$

Worked number. τ_1, τ_2 and τ_3 take 9, 8 and 8 timesteps, so $\bar{\zeta}_T = 25/3 \approx 8.33$.

Source. Ma et al. (2020) define the service time as "the average number of timesteps ... needed to finish executing each task after it was added to the task set" and use it to evaluate a MAPD algorithm. The thesis uses it unchanged. *Own part:* none.

Role. The primary result. Adaptive storage beats fixed storage under a controller π when both runs keep up and $\bar{\zeta}_T$ is lower by more than the variation between seeds (Section 4).

3.3 Question 3. Why did the result change?

These four measures explain a change in service time. They are reported next to it and never added to it.

Blocked time W_T

Question. How much time do agents lose standing still once they have a task?

What it counts. From assignment to delivery, every timestep in which the task's agent does not move, either because it chose to wait or because conflict resolution turned its move into a wait. The count is averaged over the finished tasks, like service time.

Formula.

$$W_T = \frac{1}{|\mathcal{C}_T|} \sum_{\tau_j \in \mathcal{C}_T} \sum_{t=t_j^{\text{assigned}}}^{t_j^{\text{finish}}-1} \omega_j(t) \qquad (\texttt{eq:waiting}, \text{ unchanged}).$$

Worked number. τ_1, τ_2 and τ_3 are blocked 0, 2 and 1 times, so $W_T = 3/3 = 1$. The stuck task τ_4 is not counted here, because question 1 already catches it.

Source. No paper in the folder reports blocked time. *Own part:* the whole measure. It is built from the override flag of the conflict-resolution operator, which the Method already logs.

Role. Explains service time, and answers the waiting part of RQ3.

Crowding $\bar{\delta}_T$

Question. How crowded was the space around the agents?

What it counts. Each agent's congestion feature $\delta_i(t)$, the share of its field of view taken by other agents, averaged over every agent and every timestep.

Formula.

$$\bar{\delta}_T = \frac{1}{mT} \sum_{t=0}^{T-1} \sum_{i=1}^{m} \delta_i(t) \qquad (\delta_i(t) \text{ from } \texttt{eq:congestion}).$$

Worked number. At $t = 11$ in `observation.png`, a_1 reads $\delta_1 = 2/5 = 0.4$. If the four agents read 0.4, 0.2, 0.4 and 0.2 throughout, then $\bar{\delta}_T = 0.3$. On average, 30% of the space around an agent was taken.

Source. The per-agent density is the one CRAMP (Pham and Bera, 2024) computes, the agents in the field of view divided by the free space there. CRAMP uses it only inside its reward. *Own part:* averaging it into a reported outcome.

Role. Explains blocked time. Crowding without blocking costs nothing, as with a convoy that never stops. Blocking without crowding points to the controller rather than to the traffic. Answers the congestion part of RQ3.

Traffic share of the busiest edges Γ_T

Question. Does the traffic pile up on a few aisles?

What it counts. The share of all edge traversals that falls on the busiest 10% of the edges. If traffic is spread evenly, the busiest 10% of edges carry about 10% of it. The higher the share, the more the traffic piles up.

Formula. With $\mu_T(e)$ the traversals of edge e over the horizon and E_{10} the $\lceil 0.1\,|E| \rceil$ edges with the largest $\mu_T(e)$,

$$\Gamma_T = \frac{\sum_{e \in E_{10}} \mu_T(e)}{\sum_{e \in E} \mu_T(e)}.$$

Worked number. The warehouse has 36 edges, so the busiest 4 count. In Figure 2, each panel has 72 traversals and the busiest 4 edges carry 8, 24 and 36 of them, so Γ_T is 11%, 33% and 50%.

Source. No paper in the folder reports traffic concentration. *Own part:* the whole measure. It replaces the entropy H_T and the index C_T. Every edge always counts, used or not, so the corridor problem of Decision A in version 2 does not arise.

Role. Answers the concentration part of RQ3, the risk that feedback between storage and routing funnels traffic onto a few aisles.

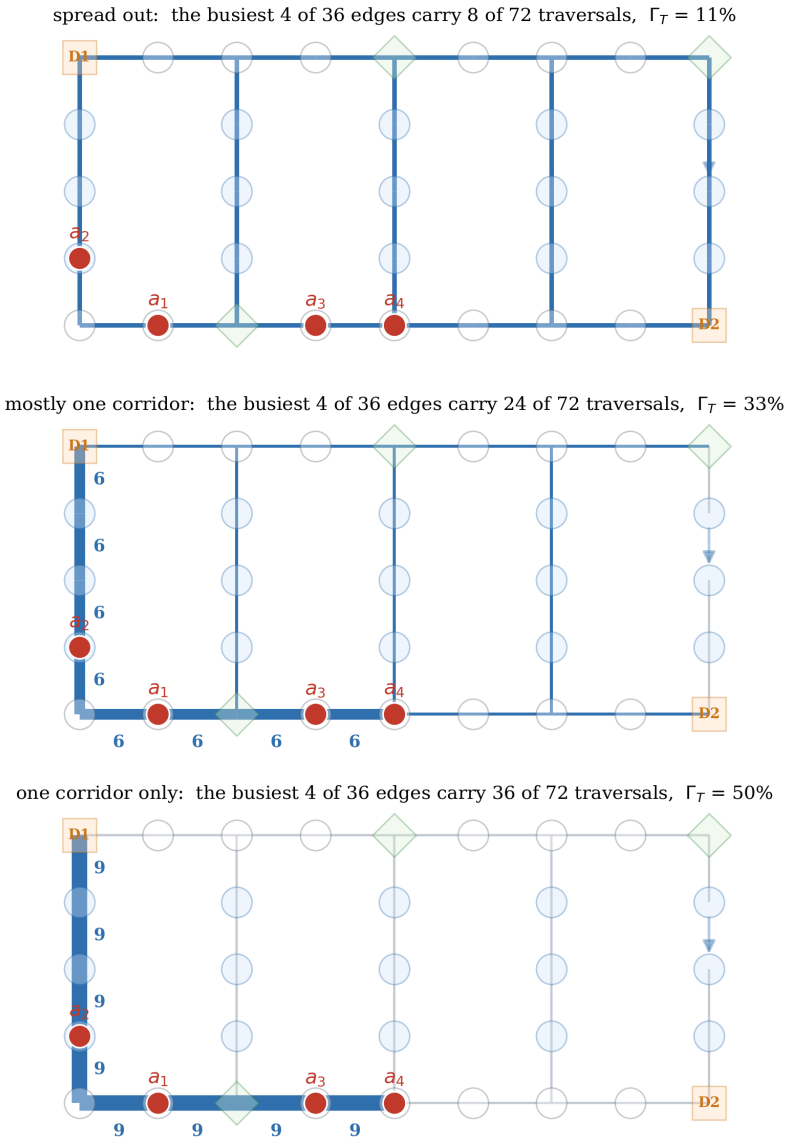


Figure 2: The concentration figure scored by the share of the busiest 4 of 36 edges, on the warehouse and agents of `observation.png`.

Movement cost per finished task $\bar{c}_T$

Question. How much effort does one delivered task take?

What it counts. The one-step cost $\hat{c}$ of every agent at every timestep, summed and divided by the number of finished tasks. Standing still costs c_{wait}, so it counts too (E1 from version 2).

Formula.

$$\bar{c}_T = \frac{1}{|\mathcal{C}_T|} \sum_{i=1}^{m} \sum_{t=0}^{T-1} \hat{c}\big(\ell_i(t), \ell_i(t+1)\big) \qquad (\texttt{eq:movementcost},\ \text{unchanged}).$$

Worked number. If the four agents make 72 moves of cost 1, wait 8 times at $c_{\text{wait}} = 0.5$ and finish 12 tasks, then $\bar{c}_T = (72+4)/12 \approx 6.33$. These cost values only serve the example.

Source. CRAMP reports "total moves", the number of non-idle actions of all agents (Pham and Bera, 2024). *Own part:* weighting each step by its cost, since edges in this graph

differ in length, and dividing by the finished tasks, since a lifelong run has no fixed number of tasks.

Role. Answers the movement cost part of RQ1. Adaptive storage can shorten trips, and this is where it shows.

3.4 Question 4. What does deciding cost?

Runtime per timestep κ_T

Question. How long does the controller take to choose the next joint action?

What it counts. The wall-clock time of assignment and routing together at each timestep, averaged over the horizon. How it is measured is fixed in `sec:impl:cost`.

Formula.

$$\kappa_T = \frac{1}{T}\sum_{t=0}^{T-1} \kappa(t) \qquad (\texttt{eq:runtime},\ \text{unchanged}).$$

Worked number. If deciding takes 2 ms at every timestep, $\kappa_T = 2\,\text{ms}$.

Source. Ma et al. (2020) report runtime per timestep for every algorithm and ask for it to stay below one second for real-time lifelong operation. *Own part:* none.

Role. Reported. It is in seconds, so it is never compared with service time.

4 The research question, formally

The decision problem becomes one sentence. Choose the storage rule and the controller with the shortest expected service time among those that keep up. Formally,

$$(F^\star, \pi^\star) \in \arg\min_{F\in\mathcal{F},\,\pi\in\Pi} \mathbb{E}\big[\bar{\zeta}_T(F,\pi)\big] \quad \text{subject to keeping up and the constraints of the model.}$$

The expectation is over the order stream and the random seeds. The study estimates it by the mean over evaluation seeds.

The thesis does not solve this problem. It compares fixed and adaptive storage under each controller, so `eq:rqformal` reads

$$\mathbb{E}\big[\bar{\zeta}_T(F,\pi)\big] < \mathbb{E}\big[\bar{\zeta}_T(F_{\text{fix}},\pi)\big] \quad \text{by a margin exceeding seed variation, with both runs keeping up.}$$

Robustness. The main RQ also asks about robustness. Under this scheme it needs no extra measure. A gain is robust if it holds in most seeds and keeps its sign as the load grows (RQ4). Lau and Sengupta (2022) report each measure as a mean with its standard error over five seeds, which is the reporting style proposed here. The example from version 2 still applies. A larger mean gain that loses in two of five seeds is less robust than a smaller gain that wins in all five.

5 What the literature measures

All papers below are in `thesis-progress/papers/` and already in `references.bib`. The table lists what each one reports and what the thesis takes from it.

Paper	What it reports	Used here
Ma et al. (2020), lifelong MAPD	service time as the measure of an algorithm, bounded service time as the condition for solving an instance, makespan, runtime per timestep	service time (Q2), the keep-up condition (Q1), runtime (Q4). Makespan is not used, see below.
Krnjaic et al. (2022), warehouse MARL	pick rate, completed order-lines per hour, as the single objective	throughput (Q1), and the case for one objective
Lau and Sengupta (2022), MAPD with MAPF and MARL	flow time and makespan for the first delivery, reward, deliveries per agent, episode time, as means with standard errors over five seeds. The paper notes that MAPF and MARL use different measures, which makes the two hard to compare.	the reporting over seeds (robustness)
CRAMP (Pham and Bera, 2024)	success rate, makespan, total moves, collision count. Crowd density is used in the reward only.	the per-agent density (crowding, Q3), total moves as the origin of movement cost (Q3)
PICO (Li et al., 2022), PRIMAL (Sartoretti et al., 2019), van Knippenberg et al. (2021)	success rate, makespan, collisions, share of solved instances	none, see below

Measures not used, and why.

- *Makespan and success rate* are one-shot MAPF measures. Every agent has one goal, and the run ends when all have arrived. A lifelong run has no moment when everything is done, so the keep-up check takes the place of success.
- *Collision count* is always zero here, because conflict resolution prevents every collision by construction. The cost of the prevented collisions shows up in blocked time instead.
- *Flow time for the first delivery* looks only at the first task of each agent, which in a lifelong run says little about the steady pattern the study compares.

6 What is removed and what it touches

Removed from the model.

- $\mathcal{J}(F, \pi)$ (`eq:functional`) and its five weights $q_\bullet$.
- The entropy H_T, the index C_T, E_T^+ and $p_T(e)$ (`eq:entropy`, `eq:concentration`), with the `[G8]` convention for a single used edge.
- The backlog as a weighted term. The backlog stays, as part of the keep-up check.

Added. Γ_T, $\bar{\delta}_T$ and the threshold f_{up}. The letter Γ is unused in the notation table, although lower-case γ is the discount factor. If that is too close, another letter is easy to pick.

Places in the thesis that change.

- The scoring section itself, rewritten as the four questions.
- `eq:decisionproblem` and `eq:rqformal`, as in Section 4.

- The Project Description's list of dependent variables.
- The scope paragraph on deadlines (`[K2]`), which says a late delivery degrades $\mathcal{J}$ and that deadlines could become a term in `eq:functional`. It would refer to service time instead.
- The opening of the Method chapter, which names $\mathcal{J}$ as a framing device.
- The instrumentation section, where H_T becomes Γ_T.
- The notation table, for the rows above.
- One new `[Axx]` tag and a `GAPS.md` entry recording the change and its reasons.

Not touched. The RL reward was already defined apart from $\mathcal{J}$, as a tool for good behaviour, so training does not change. The storage rule F and its objective do not change either.

7 What happens to the version 2 decisions

Version 2 item	In version 3
A. Normalising the entropy	No longer needed, because Γ_T counts every edge.
B. Crowding $\bar{\delta}_T$	Kept, in question 3.
C. Robustness	Seed spread and load, Section 4.
D. Stuck tasks	Settled. They fail the keep-up check in question 1.
E. Name "movement cost"	Kept, with the clause on c_{wait}.
New structure	Replaced by the four questions, in the same spirit, with a plain overview first.
Timeline figure	Kept, and now the main worked example.
New concentration figure	Kept, scored by Γ_T.
Fixes P1 to P11	P1, P2, P3, P9, P10 and P11 still apply. P4, P5 and P7 are settled by the new scheme. P6 and P8 are covered by the new list of dependent variables and the movement cost card.

8 Your decisions

Your decisions.

1. Replace $\mathcal{J}$ with the four questions, with service time as the one primary result?
2. The keep-up test is throughput of at least f_{up} times the arrival rate. Should f_{up} stay TBD in the Method tables, or do you have a value in mind?
3. The busiest 10% of edges for Γ_T, or another share?
4. Should the thesis state in one sentence which measures are borrowed and which are its own, as the cards do here?

Once you agree, the changes are applied as listed in Section 6. The figure scripts then move to `thesis-progress/scripts/images/`, with outputs copied to both `Images/` folders, and nothing is committed before you have read the result.

References

Aleksandar Krnjaic, Raul D. Steleac, Jonathan D. Thomas, Georgios Papoudakis, Lukas Schäfer, Andrew Wing Keung To, Kuan-Ho Lao, Murat Cubuktepe, Matthew Haley, Peter Börsting, and Stefano V. Albrecht. Scalable multi-agent reinforcement learning for warehouse logistics with robotic and human co-workers, 2022. URL https://arxiv.org/abs/2212.11498.

Tim Tsz-Kit Lau and Biswa Sengupta. The multi-agent pickup and delivery problem: Mapf,

marl and its warehouse applications. *arXiv preprint arXiv:2203.07092*, 2022. URL https://arxiv.org/abs/2203.07092. Version 1, submitted 14 March 2022. Accessed: 2026-06-08.

Wenhao Li, Hongjun Chen, Bo Jin, Wenzhe Tan, Hongyuan Zha, and Xiangfeng Wang. Multi-agent path finding with prioritized communication learning, 2022. URL https://arxiv.org/abs/2202.03634.

Hang Ma, Jiaoyang Li, T. K. Satish Kumar, and Sven Koenig. Lifelong multi-agent path finding for online pickup and delivery tasks. *Transportation Science*, 54(2):473–489, 2020. doi: 10.1287/trsc.2018.0826. URL https://doi.org/10.1287/trsc.2018.0826. Originally published in AAMAS 2017 proceedings; journal version accessed 2026-06-08.

Phu Pham and Aniket Bera. Optimizing crowd-aware multi-agent path finding through local communication with graph neural networks, 2024. URL https://arxiv.org/abs/2309.10275.

Guillaume Sartoretti, Justin Kerr, Yunfei Shi, Glenn Wagner, T. K. Satish Kumar, Sven Koenig, and Howie Choset. PRIMAL: Pathfinding via reinforcement and imitation multi-agent learning. volume 4, pages 2378–2385, 2019.

Marijn van Knippenberg, Mike Holenderski, and Vlado Menkovski. Time-constrained multi-agent path finding in non-lattice graphs with deep reinforcement learning. In *Proceedings of the 13th Asian Conference on Machine Learning (ACML)*, volume 157 of *Proceedings of Machine Learning Research*, 2021. URL https://proceedings.mlr.press/v157/knippenberg21a/knippenberg21a.pdf.